```
class MapController: UIHostingController<MapView> {
    convenience init(url: URL) {
        let view = MapView()
        self.init(rootView: view)
    }
}
```

Initializers in Swift

In Swift, initializers are a bit like Rails model constructors, such as `Model.new` or custom methods you define to prepare objects before saving them to the database. Swift distinguishes between two types of initializers: *designated* and *convenience*.

- *Designated initializers* are the primary entry points for creating instances of a class. Every class must have at least one designated initializer, which is responsible for fully initializing all the properties introduced by that class before calling the initializer of its superclass. Think of these as the “core” initializers.
- *Convenience initializers*, on the other hand, are secondary helpers that make creating instances easier or more flexible. They’re like Rails scopes or helper methods that simplify certain use cases without replacing the underlying logic. Convenience initializers call a designated initializer within the same class, often adding some extra processing or providing a shorthand for common configurations.

To tie it back to Rails: a *designated initializer* is like defining `initialize` in a Ruby class, while a *convenience initializer* is like writing a `self.create_from_url(url)`‘ class method to handle a specific use case.